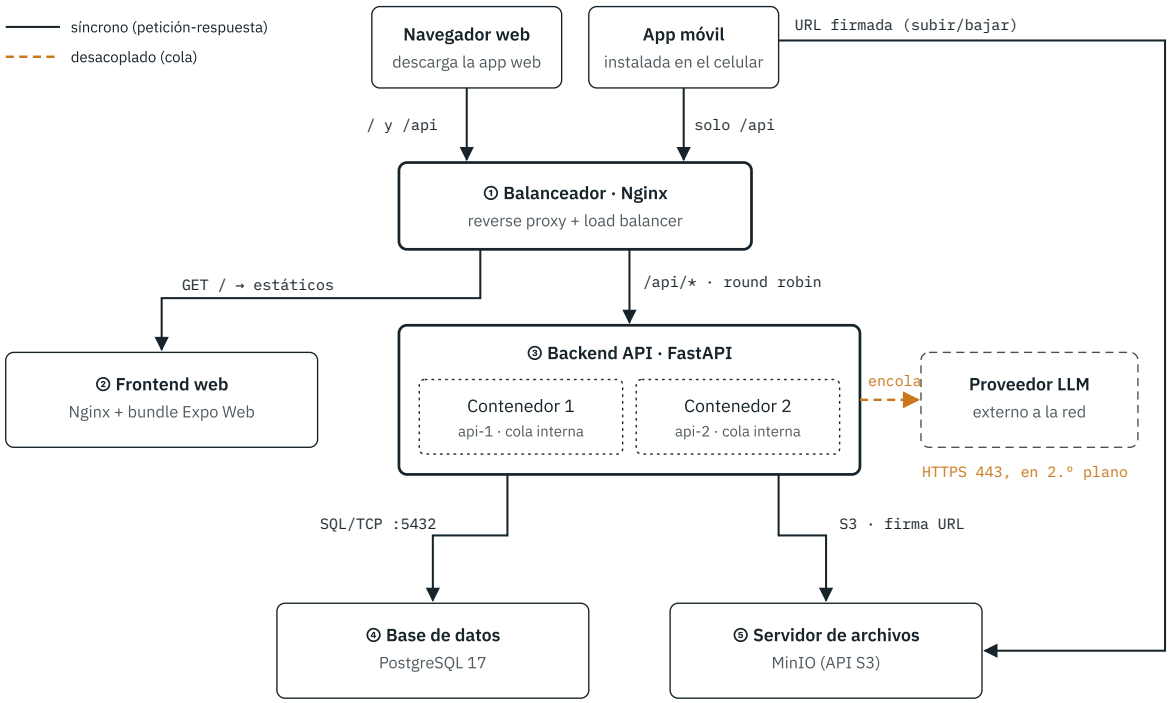
IS-803 · UNIDAD 1 · TEMA 1.4 · PROYECTO INTEGRADOR

ILIA — Arquitectura lógica y física V1

Plataforma para docentes: cliente web (Expo Web) y app móvil, API en FastAPI, PostgreSQL 17 y almacenamiento de archivos compatible con S3. Despliegue en 5 laptops de la red del laboratorio.

ENTREGABLE 1

Arquitectura lógica



Todo el tráfico de usuarios entra por ① (HTTP :80). Web y móvil salen del mismo proyecto Expo: la versión web se descarga desde ②, mientras que la app móvil ya viene instalada en el celular y solo llama a `/api`. El frontend ② entrega archivos estáticos; es el navegador quien llama a la API. Los archivos viajan directo entre el cliente y ⑤ con una URL firmada por ③, así el backend no carga con los bytes. La evaluación con IA es la única comunicación desacoplada.

Nodos y responsabilidad

NODO	TECNOLOGÍA	RESPONSABILIDAD PRINCIPAL	DEPENDE DE
① Balanceador	Nginx	Punto único de entrada. Enruta `/` al frontend y `/api/*` a los backends con round robin; saca de rotación un backend caído.	②, ③

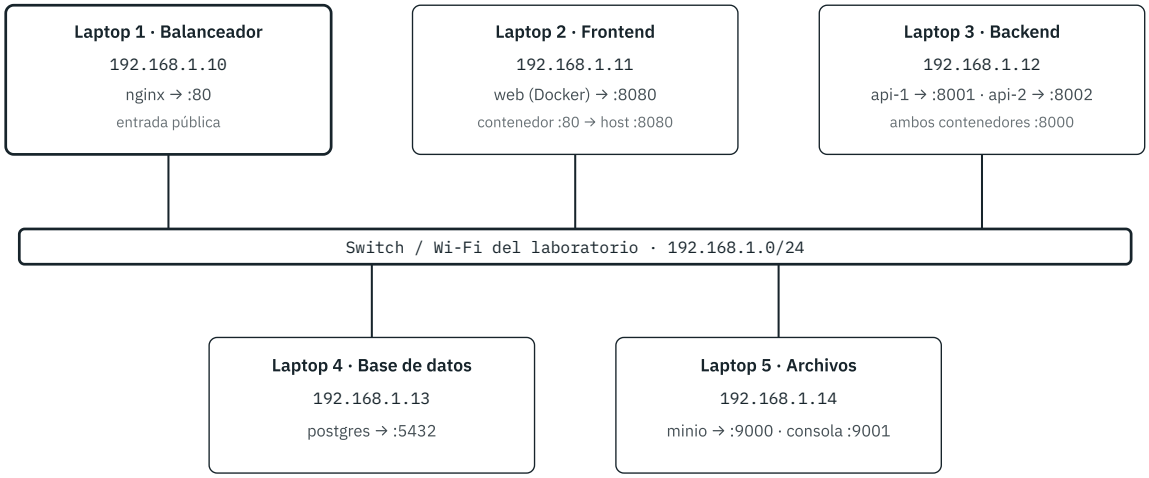
NODO	TECNOLOGÍA	RESPONSABILIDAD PRINCIPAL	DEPENDE DE
② Frontend web	Nginx + Expo Web	Servir la aplicación web (HTML/JS/CSS) y resolver rutas profundas hacia `index.html`.	Ninguno en tiempo de ejecución
③ Backend API	FastAPI + uvicorn, 2 contenedores	Autenticación (JWT en cookie), lógica de negocio, firma de URLs de archivos, cola de evaluación con IA.	④, ⑤, LLM
④ Base de datos	PostgreSQL 17	Persistencia de usuarios, cursos, entregas y evaluaciones. Migraciones con Alembic.	—
⑤ Servidor de archivos	MinIO (compatible S3)	Guardar y entregar archivos subidos (material, evidencias) mediante URLs firmadas con caducidad.	—

Cada flecha: quién llama, a quién, por qué medio y qué pasa si falla

ORIGEN → DESTINO	POR QUÉ	MEDIO	TIPO	SI FALLA
Navegador web → ①	Descargar la app web y llamar a la API	HTTP :80 (`/` y `/api`)	síncrono	El sistema no es accesible. Es el punto único de falla de la V1.
App móvil → ①	Llamar a la API (la app ya está instalada)	HTTP :80 (solo `/api`)	síncrono	La app muestra error de conexión y reintenta.
① → ②	Descargar la app web	HTTP proxy :8080	síncrono	No carga la web (502). La app móvil sigue funcionando porque solo usa la API.
① → ③	Llamadas a la API	HTTP proxy :8001 / :8002	síncrono	Si cae un contenedor, Nginx reintenta en el otro. Si caen ambos, 502 en toda la API.
③ → ④	Leer y escribir datos	TCP :5432 (protocolo PostgreSQL, asyncpg)	síncrono	`/health/ready` responde error, Nginx retira el backend y el usuario recibe 502/503. No se pierden datos ya confirmados.
③ → ⑤	Firmar URL de subida o descarga	HTTP :9000 (API S3)	síncrono	Falla solo lo relacionado a archivos; login y cursos siguen funcionando.
Cliente → ⑤	Subir o bajar el archivo real	HTTP :9000 con URL firmada	síncrono	La subida falla y el cliente reintenta. Si la URL caducó, pide otra a ③.
③ → LLM	Evaluar una entrega con IA	Cola interna + worker, HTTPS 443	desacoplado	El usuario no espera: la evaluación queda pendiente y se reintenta. Al reiniciar, el contenedor recupera las evaluaciones pendientes.

ENTREGABLE 2

Arquitectura física



Una laptop por nodo, todas en la misma subred. Las IP son de ejemplo: reemplázalas por las que asigne el router del laboratorio (conviene reservarlas por MAC para que no cambien el día de la demo).

Equipo, IP, puerto y protocolo

EQUIPO	COMPONENTE	IP	PUERTO	PROTOCOLO	ACEPTA CONEXIONES DE
Laptop 1	Nginx (balanceador)	192.168.1.10	80	HTTP	Cualquier cliente
Laptop 2	Nginx + Expo Web	192.168.1.11	8080	HTTP	Laptop 1
Laptop 3	FastAPI api-1 / api-2	192.168.1.12	8001 / 8002	HTTP (REST/JSON)	Laptop 1
Laptop 4	PostgreSQL 17	192.168.1.13	5432	TCP (protocolo PostgreSQL)	Laptop 3
Laptop 5	MinIO	192.168.1.14	9000 / 9001	HTTP (API S3) / consola web	Laptop 3 y clientes con URL firmada

Configuración del balanceador (Laptop 1)

```
upstream ilia_api {
    server 192.168.1.12:8001 max_fails=3 fail_timeout=10s;
    server 192.168.1.12:8002 max_fails=3 fail_timeout=10s;
}

server {
    listen 80;

    location /api/ {
        proxy_pass http://ilia_api/;
        proxy_next_upstream error timeout http_502 http_503;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $remote_addr;
    }

    location / {
        proxy_pass http://192.168.1.11:8080;
    }
}
```

Variables clave del backend (Laptop 3)

```
ILIA_DATABASE_URL=postgresql+asyncpg://ilia:***@192.168.1.13:5432/ilia
ILIA_CORS_ORIGINS=http://192.168.1.10
ILIA_STORAGE_ENDPOINT=http://192.168.1.14:9000
ILIA_COOKIE_SECURE=false   # solo en el laboratorio (HTTP)

# Frontend (Laptop 2), se fija al compilar:
EXPO_PUBLIC_API_URL=http://192.168.1.10/api
```

Revisa el formato exacto de `ILIA_DATABASE_URL` y el valor de `ILIA_STORAGE_PROVIDER` en `apps/api/.env.example`.

Evidencia de pruebas de conectividad

Ejecuta cada prueba desde el equipo indicado y pega la captura en la columna de evidencia.

DESDE	HACIA	COMANDO	RESULTADO ESPERADO	EVIDENCIA
Laptop 1	Todas	`ping -c 3 192.168.1.1X`	0% packet loss	captura
Laptop 1	Frontend	`curl -I http://192.168.1.11:8080`	`HTTP/1.1 200 OK`	captura

DESDE	HACIA	COMANDO	RESULTADO ESPERADO	EVIDENCIA
Laptop 1	Backend 1 y 2	`curl http://192.168.1.12:8001/health/ready` (y :8002)	200 en ambos	captura
Laptop 3	PostgreSQL	`nc -zv 192.168.1.13 5432`	`succeeded`	captura
Laptop 3	MinIO	`curl -I http://192.168.1.14:9000/minio/health/live`	`200 OK`	captura
Cliente	Sistema completo	Abrir `http://192.168.1.10` e iniciar sesión	La web carga y el login funciona	captura
Cliente	Balanceo	Detener api-1 (`docker stop`) y repetir el login	Sigue funcionando vía api-2	captura

> **Límites conocidos de la V1**
>
> - La Laptop 1 es un punto único de falla: si se apaga, nadie entra.
> - La cola de evaluación y el límite de peticiones viven en la memoria de cada contenedor. Con dos réplicas cada una cuenta por separado; para escalar en serio hace falta un almacén compartido (por ejemplo Redis).
> - El backend marca las cookies como `Secure` por defecto. Sobre HTTP simple el navegador no las guarda, así que en el laboratorio hay que desactivarlo o poner TLS en el balanceador.
> - Las migraciones (`alembic upgrade head`) se ejecutan una sola vez desde la Laptop 3 antes de levantar los dos contenedores.